

Práctica 4 – Reconocimiento de Escenas con redes convolucionales neuronales

López Martínez, Alejandro – González Sánchez, Íker

1 PREGUNTAS TAREAS OPCIONALES

4.1 Estudie la variación en rendimiento para diferentes valores del batch_size = [8, 16, 32, 64] y 50 épocas. Razone sobre los resultados observados.

Para comparar los histogramas de Accuracy y Loss finales, se utilizó la función `evaluate_generator()` sobre los conjuntos de entrenamiento y prueba. Aunque podríamos haber tomado el último valor registrado en el historial durante el entrenamiento del modelo (correspondiente a la última época), optamos por realizar la inferencia con el modelo completamente entrenado, considerando que esta opción proporciona una evaluación más confiable del rendimiento.

Como se puede observar en la Imagen 1.

En este caso, se observa que los batches de tamaños 8 y 16 presentan un rendimiento muy similar. No obstante, el tamaño 8 logró un mejor accuracy en el conjunto de validación (aunque en la gráfica es difícil distinguirlo, los valores numéricos lo confirman). Tras múltiples ejecuciones, el batch de 8 mostró un mejor rendimiento en más ocasiones, por lo que se optó por esta configuración para continuar. Sin embargo, el tamaño 16 también sería una opción válida, ya que presentó un menor loss. Al utilizar la función de pérdida Cross-Entropy, esto implica que, cuando se equivoca, el batch de 16 se desvía menos de la probabilidad correcta, lo cual es beneficioso para el entrenamiento. A pesar de esto, al evaluar cuál modelo es mejor, consideramos que el accuracy final es el indicador más representativo.

En la Imagen 2 comparamos los cuatro tamaños de batch a lo largo de las épocas. Observamos que el batch de tamaño 8 converge significativamente más rápido que los demás en el conjunto de entrenamiento, tanto en términos de accuracy como de loss. Cabe señalar que el loss y el accuracy no tienen una relación directa de 1:1, ya que la función de pérdida no se basa únicamente en si las predicciones son correctas, como se explicó anteriormente.

Esta misma tendencia se refleja en el conjunto de validación, donde el batch de tamaño 8 también converge ligeramente antes que las demás configuraciones. Este resultado nos sorprende, ya que, según la página 11 de la teoría sobre el entrenamiento de CNNs (diapositiva 22), un tamaño mayor de mini-batches debería converger más rápido y de manera más estable. Sin embargo, en este

caso, el loss del batch de tamaño 64 disminuye mucho más lentamente y ni siquiera llega a converger, lo que podría explicar su peor desempeño.

Creemos que esto podría deberse a que los batches más grandes realizan menos ajustes por iteración (a mayor tamaño de batch, menos pasos por época), lo que podría impedirles completar el proceso de ajuste en solo 50 épocas.

Finalmente, el comportamiento del loss en validación para el batch de tamaño 8 es particularmente llamativo. Esto podría deberse a que, al permitirle entrenar durante 50 épocas, los batches de tamaño 8 realizan ajustes excesivos, lo que lleva a que la red gane confianza en sus clasificaciones en el conjunto de entrenamiento. Esta sobreconfianza puede ser la causa del aumento en el loss, al reflejar una mayor certeza en etiquetados incorrectos.

4.2 Estudie la variación en rendimiento para diferentes tamaños de la imagen de entrada ([32x32, 64x64, 128x128, 224x224]) durante 50 épocas. Elija un valor de batch_size acorde a sus conclusiones en la pregunta anterior 4.1. Razone porque obtiene los resultados que observa.

Para este experimento, se ha optado por el batch de tamaño 8 debido a las razones previamente expuestas: converge más rápido y alcanza un mayor accuracy.

También observamos un considerable overfitting en estas pruebas. Como se observa en la Imagen 3, en cuanto al tamaño de las imágenes, todos los valores mayores a 64 píxeles ofrecen resultados similares, aunque el tamaño 224 destaca con un 3% más de accuracy comparado con los demás (según los valores numéricos). Sin embargo, al analizar el loss final, los resultados son muy parecidos entre los diferentes tamaños. La función de pérdida se minimiza casi hasta 0, lo que indica que un mayor tamaño de imagen permite reducir la pérdida en menos épocas. Esto podría ser un indicador de overfitting, ya que en algunos casos el loss de validación aumenta.

En la Imagen 4 se presenta una comparación similar a la de la primera pregunta. Aquí se observa que el tamaño original de 32 píxeles es demasiado pequeño, ya que no logra converger en 50 iteraciones, mientras que los demás tamaños sí lo hacen mucho más rápido, especialmente el tamaño 224.

Sin embargo, al observar los resultados en validación, no

podemos confirmar totalmente esta tendencia, ya que todos los tamaños alcanzan una zona de estabilidad en cuanto a puntuación. Aun así, el tamaño de 32 píxeles es claramente el que más se queda atrás.

En cuanto al training loss, se aprecia un patrón similar al del accuracy. Los tamaños 128 y 224 convergen en muy pocas iteraciones, mientras que el tamaño 64 tarda un poco más, aunque finalmente converge. Por otro lado, el tamaño 32 no logra converger en 50 iteraciones.

La evolución del loss en validación no muestra cambios significativos, pero se destaca que el tamaño de 32 píxeles presenta las menores pérdidas. Sin embargo, al llegar a las 50 iteraciones, las diferencias entre los tamaños son mínimas. Es interesante notar que, aunque el tamaño 224 no minimiza el loss de validación más que los demás, sí converge mucho más rápido, estabilizándose a partir de las 25 iteraciones.

Para futuros experimentos, decidimos utilizar imágenes de 224 píxeles. Esta elección se debe a su rápida convergencia en comparación con los demás tamaños, lo cual es especialmente útil en ejercicios con un límite de 25 o 15 épocas. Además, el tamaño 224 ofrece un rendimiento superior, siendo el único competidor cercano al tamaño 64, pero este requiere más épocas para igualar su desempeño.

Aunque trabajar con imágenes de 224 píxeles implica un mayor coste computacional, la diferencia de tiempo de procesamiento no es lo suficientemente significativa como para descartarlas.

4.3 Estudie la variación en rendimiento para diferentes funciones de activación durante 25 épocas. Seleccione 3-4 opciones y, salvo la última capa con activación softmax, cambie TODAS las funciones de activación de la red en las distintas capas a las opciones seleccionadas. Como inicialización de parámetros, aplique el valor por defecto en cada capa. Utilice un valor de batch_size e img_size acorde sus conclusiones en las preguntas 4.1 y 4.2. Razone porque obtiene los resultados que observa.

En la imagen 5, con la función ReLU, vemos que muestra un rendimiento prometedor con una precisión de entrenamiento que aumenta constantemente hasta alcanzar aproximadamente 0.55. Sin embargo, la precisión de validación se estabiliza alrededor de 0.35-0.4, lo que indica un claro sobreajuste. Las pérdidas disminuyen de manera constante, especialmente en el entrenamiento, lo que refuerza la evidencia del sobreajuste del modelo.

Para la imagen 6, con la función Sigmoid, vemos que presenta el comportamiento más inestable de las tres. Tanto la precisión de entrenamiento como de validación fluctúan alrededor de 0.08-0.09, mostrando un rendimiento significativamente menor que las otras funciones. Las pérdidas se mantienen estables alrededor de 1.7, pero el modelo parece incapaz de aprender

efectivamente, mostrando un estancamiento en el aprendizaje.

En la 7 con la función Tanh, muestra un comportamiento similar a ReLU pero con menor rendimiento general. La precisión de entrenamiento alcanza 0.5 mientras que la de validación se mantiene en 0.3. También presenta signos de sobreajuste, aunque menos pronunciados que ReLU. Las pérdidas disminuyen de manera más gradual en comparación con ReLU.

La gráfica comparativa final (imagen 8) nos permite ver que ReLU tiene el mejor rendimiento en términos de precisión, mientras que Sigmoid muestra el peor rendimiento pero con pérdidas más estables. El tiempo de entrenamiento es menor para Tanh, lo que podría ser ventajoso en ciertos escenarios. Las pérdidas aumentan significativamente con Sigmoid, lo que confirma su menor efectividad para este caso específico.

Como conclusión, podemos decir, que para este caso específico, ReLU se presenta como la mejor opción a pesar de su tendencia al sobreajuste, ya que logra el mejor rendimiento general. Tanh podría ser una alternativa viable si el tiempo de entrenamiento es una prioridad, mientras que Sigmoid parece ser la opción menos recomendable debido a su bajo rendimiento y comportamiento inestable. Todas las funciones muestran algún grado de sobreajuste, lo que sugiere que podrían beneficiarse de técnicas de regularización.

4.4 Estudie la variación en rendimiento para diferentes opciones de Data Augmentation (consulta ImageDataGenerator) durante 25 épocas. Seleccione 3-4 opciones, argumente su elección y compare el rendimiento obtenido con la red original del tutorial (que no aplica Data Augmentation). Utilice un valor de batch_size e img_size acorde sus conclusiones en las preguntas 4.1 y 4.2. Razone porque obtiene los resultados que observa.

En la imagen 9, vemos que la técnica de volteo horizontal muestra un rendimiento bastante equilibrado. La precisión de entrenamiento alcanza aproximadamente 0.5, mientras que la validación llega a 0.4. Las pérdidas disminuyen de manera constante, con una brecha moderada entre entrenamiento y validación. El tiempo de entrenamiento es el más bajo de todas las técnicas, lo que la hace computacionalmente eficiente.

En la 10, vemos que la rotación muestra un comportamiento interesante con una precisión de entrenamiento que alcanza 0.45 y una validación de 0.35. Sin embargo, tiene el tiempo de entrenamiento más alto y muestra la mayor pérdida en el conjunto de prueba. La brecha entre las pérdidas de entrenamiento y validación es notable, lo que sugiere cierto grado de sobreajuste.

En al 11, vemos que el desplazamiento de ancho presenta el comportamiento más estable de todas las técnicas. Las curvas de precisión de entrenamiento y validación están más cercanas entre sí, llegando a 0.45 y 0.4

respectivamente. Las pérdidas también muestran una convergencia más cercana, lo que indica un buen balance entre ajuste y generalización.

En la 12, vemos que el rango de zoom muestra resultados similares al volteo horizontal en términos de precisión, pero con un tiempo de entrenamiento significativamente mayor. Las pérdidas son más estables en comparación con otras técnicas, y la brecha entre entrenamiento y validación es moderada, sugiriendo un buen balance en el aprendizaje.

En la imagen comparativa 13, vemos que la gráfica comparativa inicial muestra que todas las técnicas logran una precisión de entrenamiento similar (alrededor de 0.5-0.55), pero difieren en el tiempo de entrenamiento y las pérdidas. El `horizontal_flip` destaca por su eficiencia computacional, mientras que `width_shift_range` sobresale por su estabilidad en el aprendizaje.

Como conclusión, para este caso específico, el `width_shift_range` parece ser la opción más equilibrada en términos de rendimiento y estabilidad, aunque `horizontal_flip` podría ser preferible cuando los recursos computacionales son limitados. `Rotation_range`, a pesar de su alto costo computacional, no muestra beneficios significativos sobre las otras técnicas. Todas las técnicas muestran algún grado de sobreajuste, pero es menos pronunciado en `width_shift_range`, lo que sugiere que podría ser la mejor opción para mejorar la generalización del modelo.

4 CARGA DE TRABAJO

Indique brevemente la carga (en horas) de cada tarea de esta práctica. Puede utilizar como ejemplo la Tabla 1.

Tarea	Horas dedicadas (Alejandro López)	Horas dedicadas (Iker González)
Tarea 1	1,5	1,5
Memoria	4	4

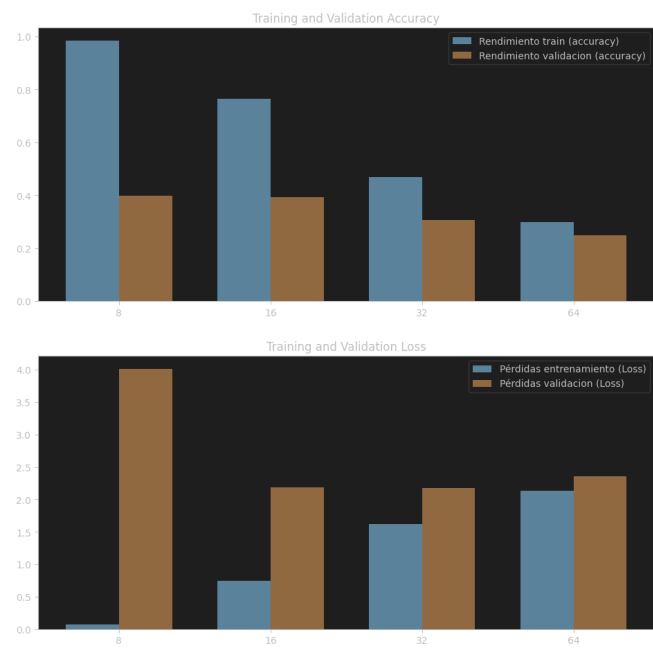


Imagen 1.

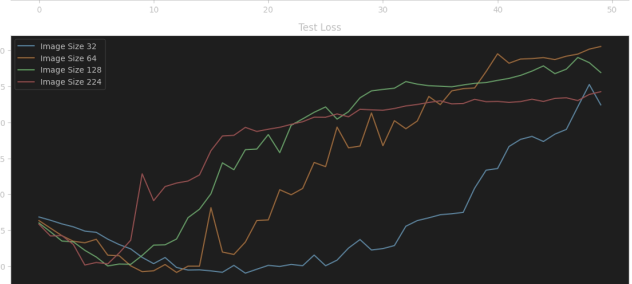
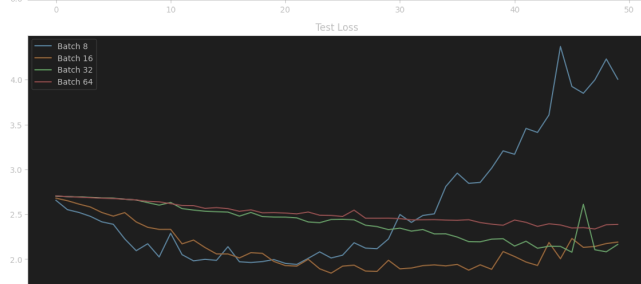
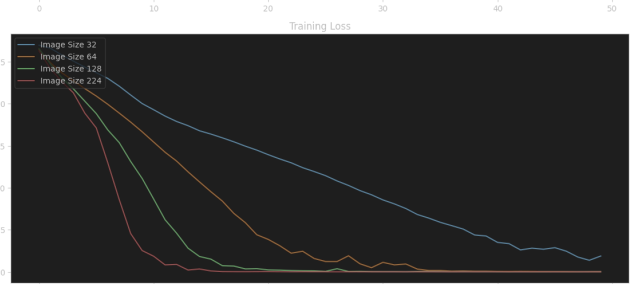
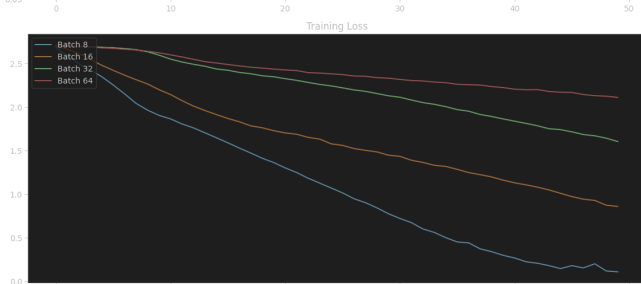
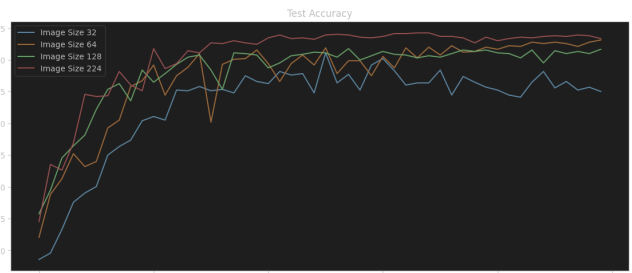
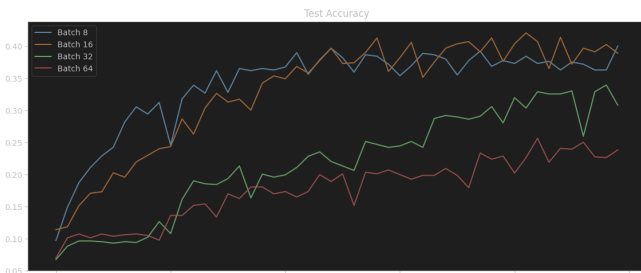
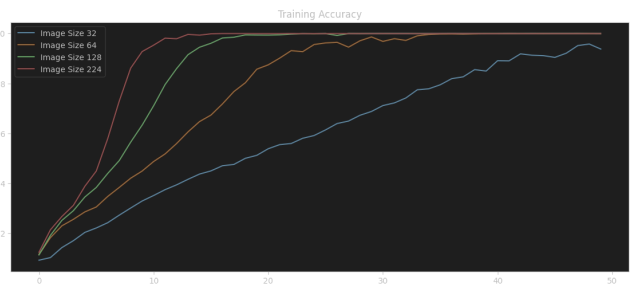
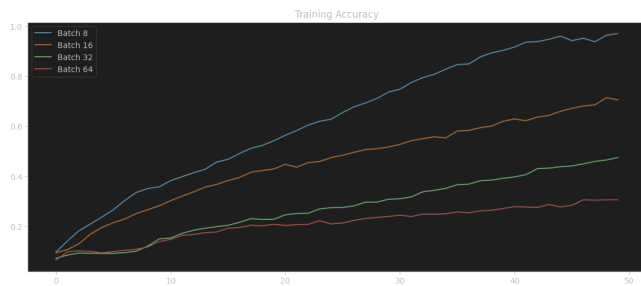


Imagen 2.

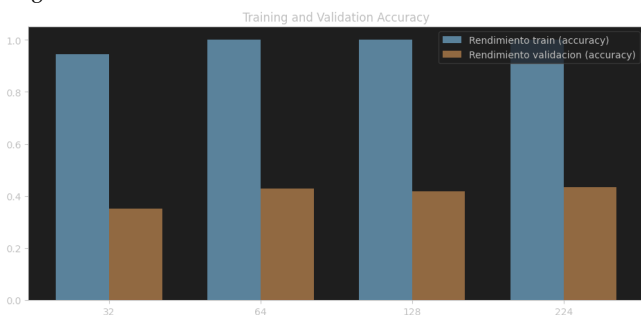


Imagen 3

Imagen 4

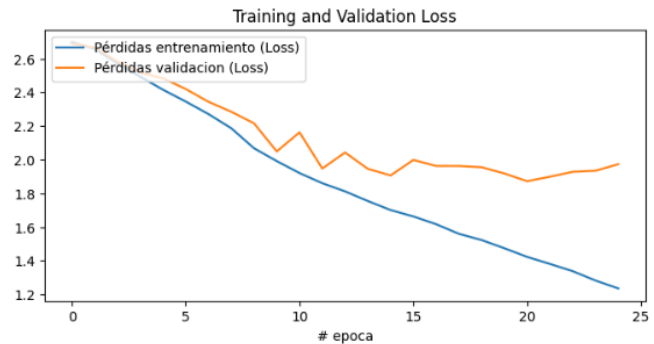
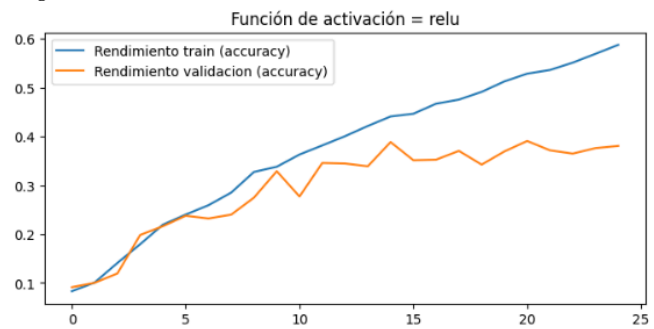


Imagen 5

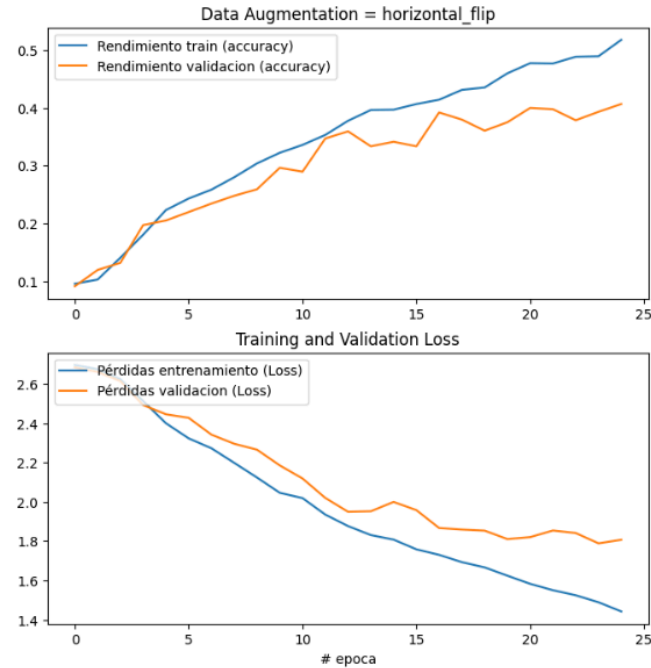
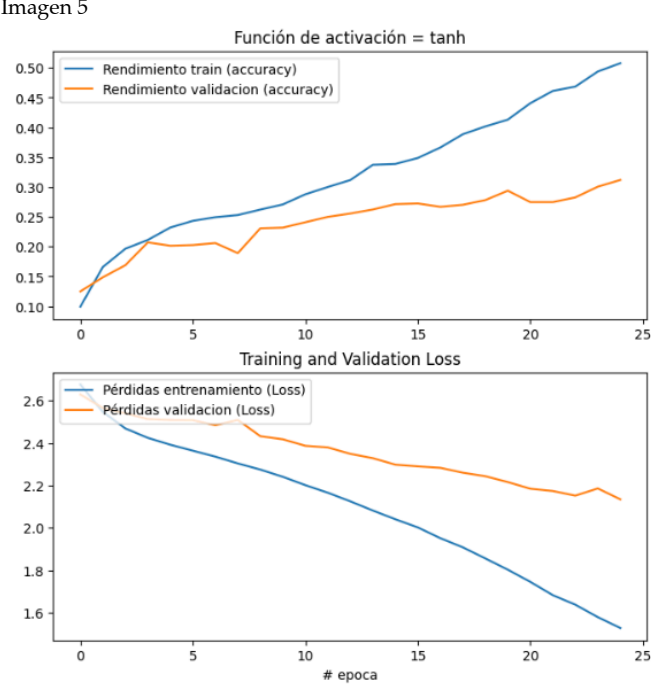


Imagen 6

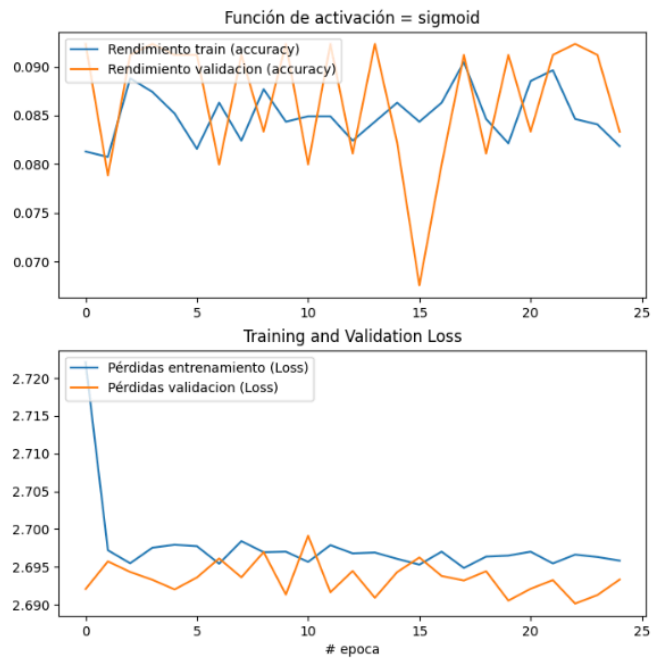


Imagen 9

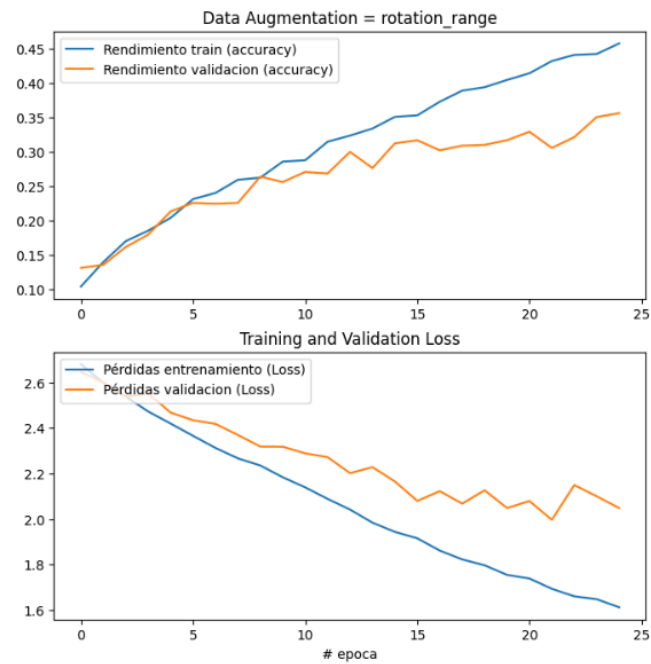


Imagen 7

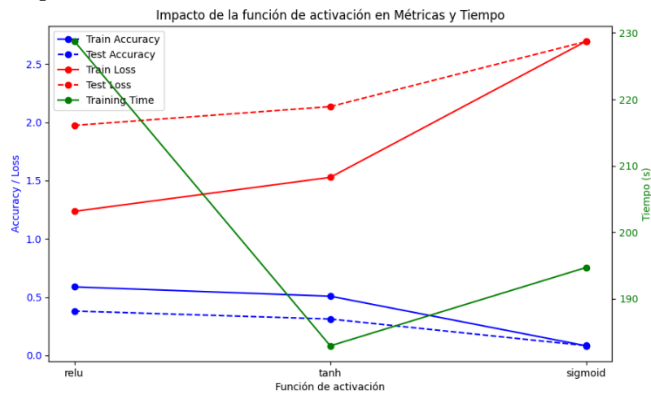


Imagen 10

Imagen 8

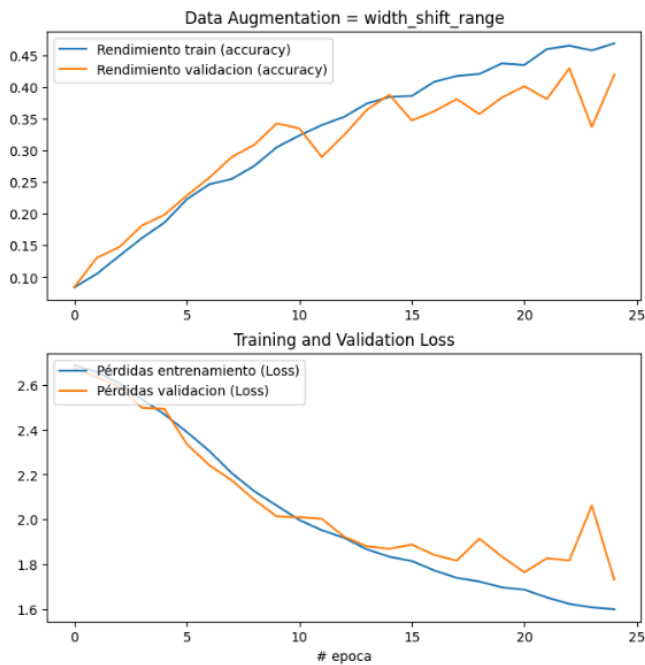


Imagen 11

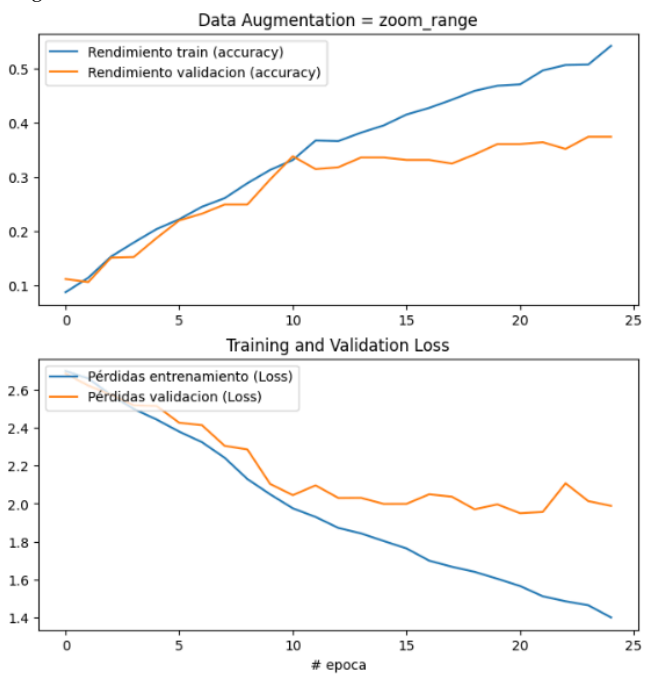


Imagen 12

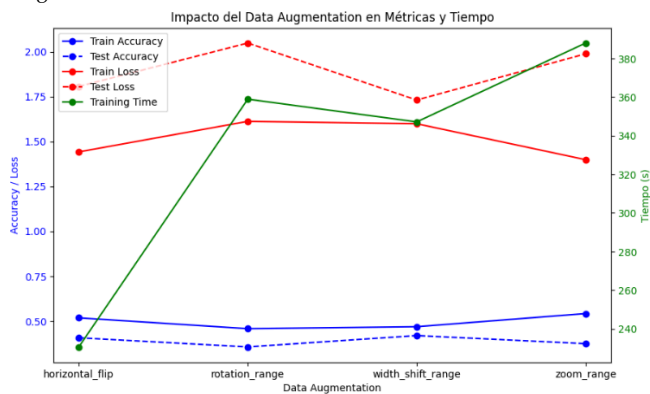


Imagen 13